

TAREA: Internet, HTTP y APIs (Lyfter)

Del Cliente al Servidor →

Explique paso a paso qué ocurre cuando usted escribe www.youtube.com en el navegador hasta que el video aparece en pantalla.

- Debe incluir en su explicación:
 - Qué hace el cliente (navegador).
 - Qué papel cumple el **DNS**.
 - Qué ocurre con la dirección IP.
 - Qué hace el servidor.
 - Cómo entra en juego el protocolo **HTTP/HTTPS**.



Puedes incluir un dibujo o diagrama que muestre cómo viaja la información entre cliente, **DNS** y servidor.

RESPUESTA:

Cuando el cliente escribe “www.youtube.com” en el navegador y pulsa Enter, ocurre lo siguiente:

1. El Cliente (El Navegador en uso)

- El navegador actúa como **cliente**.
- Primero verifica si ya conoce la dirección IP (en caché).
- Si no la tiene, necesita encontrarla → aquí entra el DNS (**resolución DNS**)

En otras palabras, el **navegador** (cliente) recibe la dirección web. Como no entiende nombres de dominio, primero verifica si tiene la página guardada en caché. Si no, inicia el proceso de resolución.

2. El papel del DNS (Domain Name System)

El DNS funciona como una “agenda telefónica de internet”

- El cliente escribes: www.youtube.com (El nombre de una pagina como “Youtube” es como un disfraz y el DNS desvela su verdadera identidad numérica)
- El DNS lo traduce a una dirección IP, por ejemplo: [142.250.xxx.xxx](https://www.youtube.com)
- Este proceso se llama **resolución DNS**
- Una vez traducida la dirección IP, este se la devuelve al navegador (Cliente)

Pasos:

1. El navegador pregunta al sistema operativo.
2. Si no está en caché, consulta a un servidor DNS.
3. El DNS responde con la IP del servidor de YouTube.

3. Conexión con el Servidor

Ahora el navegador:

- Establece una conexión con el servidor usando la IP.
- Generalmente usa el puerto **443 (HTTPS)**.
- Se crea una conexión segura mediante **TLS/SSL**

La IP es la dirección real del servidor en internet. Es necesaria porque las computadoras no entienden nombres como "youtube.com", solo números.

Con la IP, el navegador (Cliente) ya sabe a dónde enviar la solicitud, y este establece una conexión con el **servidor** de YouTube mediante el protocolo **TCP**. Inmediatamente después realiza el **handshake TLS** para crear una conexión segura **HTTPS**.

NOTA: **TLS** (Transport Layer Security) y **SSL** (Secure Sockets Layer) son protocolos criptográficos que cifran datos entre navegadores y servidores, garantizando la privacidad e integridad de la información en internet. Aunque SSL es el término tradicional, TLS es su versión moderna, más segura y actualizada. Activan HTTPS y muestran un candado en la barra de direcciones.

NOTA: **TCP** (Transmission Control Protocol), es uno de los protocolos fundamentales de Internet que permite a dos dispositivos conectarse e intercambiar flujos de datos de manera confiable, ordenada y sin errores

4. Protocolo HTTP/HTTPS

Una vez establecida la conexión segura (mediante el handshake TLS), el navegador envía una petición HTTPS al servidor.

Flujo de la petición:

El cliente (Navegador) envía una petición similar a esta:

1. GET / HTTP/2
2. Host: www.youtube.com
3. User-Agent: ...
4. Accept: text/html, ...

NOTA: Como se usa HTTPS, toda la comunicación (petición y respuesta) viaja encriptada.

El servidor (YouTube / CDN de Google):

1. Recibe y procesa la solicitud.
2. Determina qué se está pidiendo (página principal, página de video, etc.).
3. Responde enviando los recursos necesarios:
 - **HTML** → Estructura de la página

- **CSS** → Estilos visuales
- **JavaScript** → Interactividad y lógica del reproductor
- Imágenes, fuentes y otros archivos
- Para el **video**: un manifiesto (información del video) y posteriormente los fragmentos de video en streaming.

NOTA: Una **CDN** (Content Delivery Network o Red de Distribución de Contenido) es una red distribuida de servidores geográficamente dispersos que almacenan en caché copias de archivos web (imágenes, videos, HTML, JS) cerca de los usuarios. Su función principal es acelerar la carga de sitios web, reducir la latencia y aumentar la seguridad.

5. Carga y reproducción del video

El navegador interpreta el HTML y muestra la interfaz.

Al reproducir un video, el navegador hace nuevas peticiones HTTPS para obtener los fragmentos del video. El servidor los envía en streaming (generalmente mediante **CDN**), y el navegador los reproduce en pantalla. En otras palabras:

El navegador:

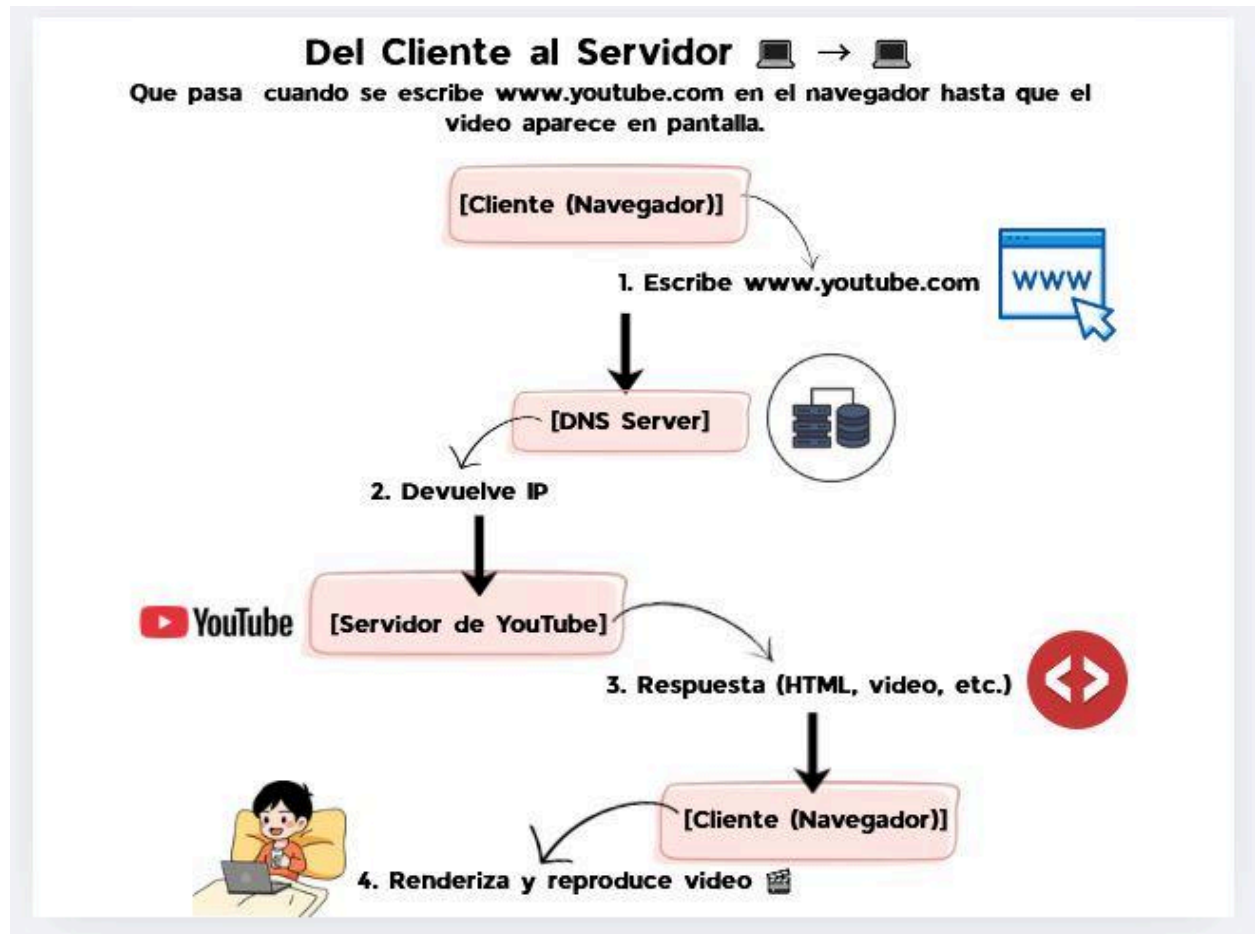
- Interpreta el HTML
- Ejecuta JavaScript
- Hace más solicitudes (por ejemplo, al hacer clic en un video)

El video no llega todo de una vez:

- Se transmite por partes (**streaming**)
- Usa protocolos como HTTP adaptativo

Resumen del flujo (General):

1. La persona/usuario escribe el dominio (URL) → Navegador consulta DNS
2. DNS traduce nombre = IP
3. Navegador se conecta vía HTTPS al servidor (Se establece conexión segura (HTTPS))
5. El cliente envía una petición
4. Servidor responde con archivos (HTML, video, etc.)
6. El navegador muestra y reproduce el contenido (Para efectos de la asignacion, el video)



Frontend y Backend en acción 🧩 (Pt. 2)

Suponga que está construyendo una app web para agendar citas médicas.

- Indique qué parte del sistema correspondería al **frontend** y cuál al backend.
- Mencione tres tecnologías posibles para cada uno.
- Explique brevemente cómo el **frontend** se comunicaría con el backend (mencione los conceptos de **API**, **HTTP** y **request/response**).

RESPUESTA:

Frontend y Backend en una App de Citas Médicas

Frontend VS Backend

- Frontend = La parte que (el usuario ve y toca).

Es la pantalla de la aplicación (como un sitio web).

Ejemplos: botones, calendario, formularios para elegir hora y doctor.

- Backend = La parte (que el usuario no ve).

Es el "cerebro" que trabaja en el servidor.

Se encarga de guardar datos, verificar que no se repitan citas, enviar correos, etc.

Tecnologías que se pueden usar

Frontend (3 opciones fáciles):

1. React.js (la más popular)
2. Vue.js
3. Angular

Backend (3 opciones):

1. Node.js + Express
2. Python (con Django o FastAPI)
3. PHP con Laravel

Cómo se comunican el Frontend y el Backend entre sí?

Se comunican usando HTTP a través de una API.

A continuación el paso a paso:

1. El usuario hace algo en la pantalla (ej: pulsa "Agendar Cita").
2. El Frontend envía una **petición (request)** al Backend.
3. El Backend recibe la petición, hace su trabajo (guarda la cita, verifica disponibilidad, etc.).
4. El Backend responde al Frontend con una respuesta (response).
5. El Frontend recibe la respuesta y muestra el resultado al usuario (ej: "¡Cita confirmada!").

Esta comunicación se hace normalmente con datos en formato JSON.

En resumen sencillo:

El Frontend es la cara bonita de la app.

El Backend es el que hace todo el trabajo importante detrás.

Se hablan entre sí mediante **peticiones y respuestas** usando Internet (HTTP + API).

3. REST vs SOAP vs GraphQL (Pt. 3) ⚙️

Tabla comparativa entre REST, SOAP y GraphQL:

REST VS SOAP VS GRAPHQL (PT. 3)				
Tipo de API	Formato de datos usado	Nivel de flexibilidad	Dificultad de implementación	Uso actual
REST	JSON (principalmente)	Media (cliente y servidor comparten control)	Baja	Alta
SOAP	XML	Baja (control fuerte del servidor)	Alta	Baja
GraphQL	JSON	Alta (cliente controla qué recibe)	Media	Media-Alta (Su auge es inminente)

¿Cuál considera más apropiada para una startup moderna que desarrolla un sistema de reservas en línea? ¿Por qué?

Partiendo del contexto, **GraphQL** puede ser la mejor opción...

Porque?

- Permite obtener **solo los datos necesarios** (ideal para apps web y móviles).
- Reduce el número de requests → **mejor rendimiento**.
- Es muy útil cuando hay **muchas entidades relacionadas** (usuarios, reservas, pagos, disponibilidad, etc.).
- Escala bien a medida que el sistema crece.

Alternativa válida:

REST también es una excelente opción si:

- El equipo busca algo más simple.
- Se necesita implementar rápido.
- No se requiere tanta personalización en las respuestas.

4. Explorando APIs con Postman

Parte 1: Selección de la API

1. Busca una API pública gratuita (sin autenticación obligatoria o con token gratuito).
 - Por ejemplo podrías utilizar:
[PokéAPI](https://pokeapi.co/)
2. Escribe en tu documento:
 - Nombre de la API
 - Breve descripción de lo que ofrece.

RESPUESTA:

Nombre de la API: PokéAPI

URL: <https://pokeapi.co/>

Descripción: PokéAPI es una API REST pública y gratuita que ofrece un extenso conjunto de datos sobre Pokémon. Permite obtener información completa de más de 1300 Pokémon, sus evoluciones, habilidades, movimientos, tipos y estadísticas. No requiere autenticación, esto la hace ideal para practicar el consumo de APIs con Postman.

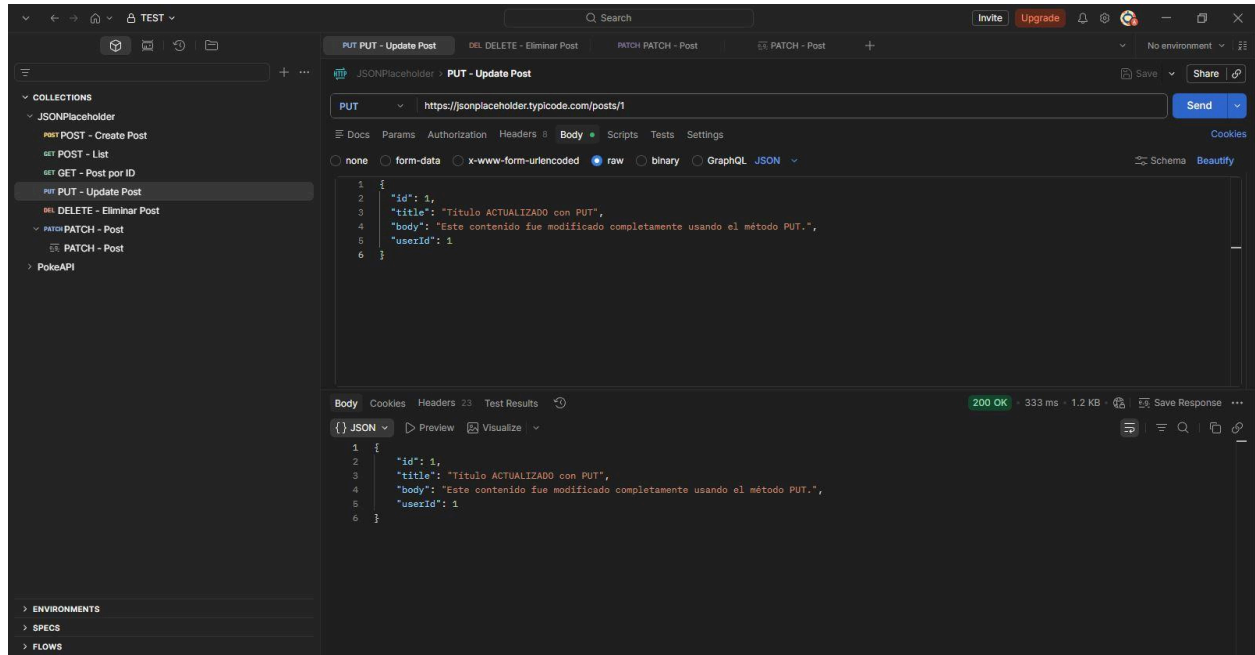
Parte 2: Configuración en Postman

1. Crea una nueva colección en Postman con el nombre de la API.
2. Agrega al menos 3 tipos de solicitudes:
 - GET → obtener información (por ejemplo, lista o detalle de un elemento).
 - POST → enviar datos (por ejemplo, crear un recurso).
 - PUT/PATCH o DELETE → actualizar o eliminar un recurso (si la API lo permite).
3. Si la API usa parámetros o tokens, configura las variables necesarias en la pestaña Environment.

Parte 3: Ejecución y análisis 🔍

1. Ejecuta cada solicitud y verifica:
 - Código de estado (200, 404, 500, etc.).
 - Cuerpo de la respuesta (**JSON** o texto).
 - Headers relevantes (por ejemplo: **Content-Type**, **Authorization**).
2. Guarda las respuestas en tu colección.

RESPUESTA:



Parte 4: Explicación técnica 🧠

Crea un archivo **README.md** (o documento de texto) que incluya:

1. Descripción general de la API elegida.
2. Explicación de cada solicitud:
 - Método HTTP usado.
 - Endpoint.
 - Parámetros o cuerpo de la solicitud.
 - Descripción breve de la respuesta.
3. Ejemplo de código o fragmento **JSON** con la respuesta.
4. Qué aprendiste del proceso.

RESPUESTA:

<https://github.com/imkerberoz/exploration-postman-jsonplaceholder/pull/1>

Parte 5: Reflexión final 🚫

En 1 o 2 párrafos, responde:

¿Qué aprendiste sobre el funcionamiento de las APIs?

¿Cómo te ayudó Postman a entender la comunicación entre cliente y servidor?

RESPUESTA:

¿Qué aprendiste sobre el funcionamiento de las APIs?

Con esta tarea aprendí mucho sobre cómo funcionan realmente las APIs. He logrado entender que las APIs son un puente que permite que las aplicaciones se comuniquen entre sí, independientemente de su lenguaje. Anteriormente ya había escuchado hablar de ellas y pensaba que eran más complicadas, pero ahora veo que básicamente se trata de hacer peticiones (preguntas) usando métodos como GET, POST, PUT o DELETE, (métodos que utilizamos a diario con el uso de aplicaciones y los browsers, pero esto lo hacemos de una forma más estética y más “user-friendly”) y recibir respuestas del servidor. Me gustó mucho ver la diferencia entre PokéAPI (que solo permite consultar información) y JSONPlaceholder (que permite crear, modificar y eliminar datos). Ahora comprendo mejor el significado de los códigos de estado y la importancia de enviar los datos correctamente en formato JSON.

¿Cómo te ayudó Postman a entender la comunicación entre cliente y servidor?

Postman me ayudó muchísimo a entender la comunicación entre el cliente y el servidor. Poder escribir la solicitud, enviarla y ver inmediatamente la respuesta (el código, los headers y el JSON) hizo que todo fuera mucho más real y fácil de entender. Es como si pudiera “ver por dentro” cómo se comunican las aplicaciones. En principio ya no me siento tan perdido como cuando había escuchado hablar de APIs anteriormente, y siento que he ganado más confianza para seguir aprendiendo sobre desarrollo web, con más práctica estoy seguro que puedo “agarrarle el toque”! 😊